```
'1,2,0,2,8,0,1,3'
```

You already have a `split` function to split comma-separated strings, but you'll need a way to add up the numbers returned. To do this you'll need a `sum` function that works as follows:

```
numbers = [1,2,0,2,8,0,1,3]
sum numbers
# 17
```

You haven't written the function yet, but that's how it should work.

Writing accumulate

One way to add the numbers in an array is by keeping a total and adding each one in turn:

```
sum = (numbers) ->
  total = 0
  for number in numbers
    total = total + number
  total

sum [1..5]
# 15
```

Use a for comprehension to accumulate the total by adding each number in the range to the total as it goes through.

Suppose you want to multiply the numbers instead of sum them. Do you write another function?

```
multiply = (initial, numbers) ->
  total = initial or 1
  for number in numbers
    total = total * number
  total
```

Can't start with a total of 0 if multiplying: 0x1x2x3x4x5 is 0.

The `multiply` function looks almost exactly like the `sum` function! Now suppose you want to flatten an array of arrays so that `[[1,2],[2,2],[4,2]]` will become `[1,2,2,2,4,2]`. How do you do that? You *could* write yet another function that looks just like the last two: